

AI반도체특론

개인 과제 보고서

2025314102 이다인

1. 과제 수행 내용 및 연구 방법론

본 실습에서는 에지(Edge) 디바이스 환경에서 가위바위보 이미지 인식 시스템을 구축하고, 모델 경량화가 하드웨어 아키텍처에 미치는 실질적인 영향을 규명하기 위해 두 가지 측정을 수행하였다. 단순한 소프트웨어 구동을 넘어 시스템 레벨의 논리적 타당성을 확보하기 위해 실험을 설계했다.

- 카메라 기반 실시간 측정:** 원본 SSD 모델을 실제 Raspberry Pi 카메라 모듈과 연결하여 실시간 프레임 단위 메트릭을 기록했다. 초기 카메라 설정, TFLite 인터프리터 준비, CPU 주파수 유동성으로 인한 오차를 배제하고자 10초의 워업(Warmup) 구간을 두었으며, 안정화된 이후의 60초간의 측정 구간 데이터만 평균 성능 계산에 활용하였다. 시스템 지표는 1초 주기로 정밀 샘플링하였다.
- Headless 벤치마크 및 하드웨어 카운터 측정:** 원본 모델과 경량화된 SSD-Lite(FP32, INT8) 모델을 동일 조건에서 비교하기 위해 입력값 분리 후 Headless 추론을 수행했다. 모델 간 열 축적 피드백을 차단하고자 실행 사이에 30초의 쿨다운(Cooldown) 시간을 강제하였고, perf stat 기반 측정 시 시스템 노이즈를 격리하기 위해 taskset -c 0 명령어로 CPU 코어를 0번에 고정했다.
- 전력 대리 지표(Power Proxy Metric) 설정:** 물리적 전력계 없이 발열 경향성을 파악하기 위해 Linux 커널 인터페이스(/sys/class/thermal/thermal_zone0/temp)를 통해 SoC 온도를 주기적으로 샘플링하여 산술 평균한 soc_temp_mean_c를 정의하고, 이를 전력 소비 패턴의 대리 지표로 활용하였다.

2. 실험 결과 및 아키텍처 지표 분석

추론 런타임 및 하드웨어 성능 카운터를 분석한 결과, 모델 경량화가 디바이스 자원 전반에 이점을 가져옴을 확인하였다.

모델명	모델 크기 (MB)	평균 지연 시간 (ms)	RSS 메모리 평균 (MB)	SoC 평균 온도 (°C)
Original SSD	10.96	669.83	85.73	60.92 (최대 65.6)
SSD-Lite FP32	0.11	11.26	52.54	56.29
SSD-Lite	0.03	1.30	52.85	54.36

INT8				
------	--	--	--	--

TFLite Headless 벤치마크 결과

SSD-Lite 경량화 모델은 원본 대비 **99% 이상의 용량 감소**, 약 55배의 추론 속도 향상, 33MB 수준의 메모리 절감을 달성했다. 원본 모델 구동 시 SoC 온도가 최대 65.6°C까지 치솟았던 반면, 경량 모델은 58°C 이하의 안정적인 열 프로파일을 유지하였다.

모델명	추론당 사이클 수 (Cycles)	추론당 명령어 수 (Instr.)	IPC	캐시 미스율	분기 예측 미스율
SSD-Lite FP32	7.75 M	18.35 M	2.37	1.20%	0.56%
SSD-Lite INT8	3.73 M	7.78 M	2.08	2.38%	1.25%

하드웨어 성능 카운터 분석 (perf stat 단일 코어 고정)

양자화된 INT8 모델은 FP32 대비 사이클 수 약 52%, 명령어 수 약 58%가 감소하여 절대적인 연산량이 크게 줄었다. 그러나 아키텍처 관점에서 IPC, 캐시 미스율, 분기 미스율은 INT8이 오히려 불리한 지표를 보였다. 이는 데이터 타입 자체의 결함이 아니라 런타임 최적화 레이어의 개입 여부 때문으로 보인다. TFLite 런타임은 FP32 모델 실행 시 XNNPACK delegate를 활성화하여 ARM NEON SIMD 벡터화 및 최적화된 메모리 접근 패턴을 적용하지만, 완전 양자화된 INT8 모델은 XNNPACK이 지원되지 않아 레퍼런스 커널(Reference Kernel)로 폴백(Fallback)될 수 있다. 결과적으로 멀티코어 환경에서는 XNNPACK의 병렬화 이점으로 인해 FP32(1.26ms)가 INT8(1.31ms)보다 미세하게 빠르게 측정되는 현상이 발생하였으며, 이를 통해 실제 디바이스 배포 시에는 모델 알고리즘뿐만 아니라 하위 런타임 프레임워크의 아키텍처 최적화 레이어까지 통합적으로 고려해야 함을 실험적으로 증명했다.

3. 학습을 통한 성장

컴퓨터 아키텍처를 전공하고 뉴럴 네트워크 하드웨어 가속 구조를 연구하는 입장에서, 본 실습은 지식과 감각을 소프트웨어 상위 레이어에서 물리 하드웨어 하부 레이어까지 관통하는 계기가 되었다. 그동안 아키텍처 시뮬레이터나 추상적인 알고리즘 레벨에서 머신러닝 워크로드를 다룰 때는 메모리 병목이나 전력 효율성을 수치적으로만 인지하곤 했다. 그러나 런타임 단에서 하드웨어 카운터를 직접 추출하고, 에지 디바이스의 열적 거동(Thermal behavior)을 추론 인터벌과 연계하여 분석해보며 실제 하드웨어가 직면한 실생활 이슈를 깊이 이해하게 되었다.

하드웨어와 소프트웨어를 잇는 아키텍처 연구자는 시스템 전체를 통합적인 시각에서 바라보아야 합니다. 본 과목을 통해 실험 데이터를 베이스라인부터 체계적으로 파싱하고 아키텍처적 인과 관계를 추론하는 분석적 방법론을 체득할 수 있었다. 이번 경험은 향후 온칩(On-chip) 메모리의 효율적 관리 기법, 하드웨어·소프트웨어 코디자인(Co-design), 자원 제약적 환경에서의 가속기 신뢰성 연구 등 본인의 세부 연구를 전개하는 데 토대가 될 것이다. 나아가, 하드웨어 제약 조건을 돌파하는 실무적인 아키텍처 설계 엔지니어로 성장하기 위한 경험을 할 수 있었다.

